

Riemann_Lab — Complément au PDF méthode : Git, priorités Phase 3 et suite du soir

Ce document complète le PDF « 04_methode_scripts_post_audit_et_remarques_materiel_hprzeta ». Il ajoute les constats du mini-audit Git, les décisions à ne pas oublier, les actions immédiates avant la phase priorité 3, puis le déroulé prévu pour ce soir.

Statut global

Tous les éléments critiques sont maintenant réunis : audit système, résumé d'archive, détail texte système, mini-audit Git, PDF de méthode et organisation Windows dans C:\PerSoTestArmel\Zeta. Le point majeur restant est de traiter proprement les fichiers non suivis, sauvegarder les branches, puis tester la Phase C.

1. Complément depuis le mini-audit Git

- Branche active au moment du mini-audit : Riemann_Lab_IA, à jour avec origin/Riemann_Lab_IA au commit 4d6617c, message « docs: MAJ index ».
- Branches locales confirmées : main, Riemann_Lab_IA, Riemann_Lab_C, Riemann_Lab_Test.
- Riemann_Lab_C est synchronisée avec origin/Riemann_Lab_C au commit db147dc, message « docs(claude): sync from GitHub, add double-update rule ».
- Riemann_Lab_Test est en avance de 14 commits sur origin/Riemann_Lab_Test : cette branche doit être sauvegardée avant tout nettoyage ou réinstallation.
- Le remote Git est git@github.com:hprzeta/Riemann_Lab.git en fetch et push.
- Le dépôt contient des fichiers non suivis, notamment les fichiers d'audit et l'archive tar.gz de 12 Go dans scripts/. Cette archive ne doit jamais être committée.

2. Constats critiques à ajouter au rapport méthode

Sujet	Constat	Décision
Archive audit 12 Go	Présente dans scripts/ comme fichier non suivi.	Déplacer hors dépôt vers ~/hprzeta-system-audit/full_archives et ignorer via .gitignore.
Fichiers assistant_pack	Présents dans scripts/ comme fichiers non suivis.	Déplacer vers ~/hprzeta-system-audit/final_texts ou dossier Windows 8-audit-system.
.mcp.json	Fichier non suivi.	Ne pas committer sans revue ; ajouter au .gitignore si spécifique local.
Riemann_Lab_Test	En avance de 14 commits.	Créer un git bundle --all avant toute réinstallation.
Riemann_Lab_C	Contient les fichiers Phase C et le .so.	Tester reproductibilité : make clean, make, test_illinois.py, benchmark_illinois.py.
compute v4	La branche C contient compute_zeros_v4.py et non compute_zeros_v4_1.py.	Corriger la nomenclature des prochains rapports/scripts.

3. Confirmation Phase C

Le mini-audit Git confirme que Riemann_Lab_C contient les fichiers C attendus : Makefile, benchmark_illinois.py, illinois_mpfr.c, illinois_mpfr.h, illinois_mpfr.so, test_illinois.py, z_function.c et z_function.h. C'est une progression importante : la prochaine phase n'est plus seulement de créer le module C, mais de vérifier que le binaire .so est reproductible et validé.

Décision : ce soir, la priorité 3 doit commencer par la sauvegarde Git, puis le nettoyage des fichiers d'audit hors dépôt, puis seulement le test Phase C.

4. Commandes immédiates à garder pour ce soir

4.1 Déplacer les fichiers d'audit hors du dépôt

```
cd ~/projet_zeta
mkdir -p ~/hprzeta-system-audit/final_texts
mkdir -p ~/hprzeta-system-audit/full_archives

mv scripts/hprzeta_system_audit_2026-05-27_01-40-50.tar.gz ~/hprzeta-system-audit/full_archives/ 2>/dev/null || true

mv scripts/assistant_pack_audit_*.txt ~/hprzeta-system-audit/final_texts/ 2>/dev/null || true

mv scripts/audit_extract_text_v2 ~/hprzeta-system-audit/final_texts/ 2>/dev/null || true
```

4.2 Ajouter les exclusions au .gitignore

```
cat >> .gitignore <<'EOF'

# Audit système local hprzeta — ne pas versionner
scripts/hprzeta_system_audit_*.tar.gz
scripts/hprzeta_system_audit_*.zip
scripts/assistant_pack_audit_*.txt
scripts/audit_extract_text*/
.mcp.json
EOF
```

```
git status
```

4.3 Créer une sauvegarde Git complète

```
cd ~/projet_zeta
mkdir -p ~/hprzeta-git-backups

git bundle create ~/hprzeta-git-backups/Riemann_Lab_all_branches_2026-05-27.bundle --all
git branch -vv > ~/hprzeta-git-backups/branches_vv_2026-05-27.txt
git log --oneline --decorate --graph --all -100 > ~/hprzeta-git-backups/git_log_all_2026-05-27.txt

git status > ~/hprzeta-git-backups/git_status_after_cleanup_2026-05-27.txt
ls -lh ~/hprzeta-git-backups
```

5. Priorité 3 — test Phase C prévu ce soir

Objectif : vérifier que le module C/libmpfr est présent, compilable, testable et exploitable par compute_zeros_v4.py.

```
cd ~/projet_zeta
```

```
git checkout Riemann_Lab_C
git status
```

```
cd src/calculs/optimisation/c_modules
ls -lah
make clean
make
python3 test_illinois.py
python3 benchmark_illinois.py
```

Résultats attendus :

- Compilation propre de illinois_mpfr.so depuis illinois_mpfr.c et z_function.c.
- test_illinois.py valide les premiers zéros LMFDB avec tolérance acceptable.
- benchmark_illinois.py produit un temps C/libmpfr comparé au fallback Python/mpmath.
- Si un échec apparaît, conserver la sortie complète du terminal dans phase_c_test_log.txt.

```
# En cas de test ce soir, capturer le log complet :
cd ~/projet_zeta/src/calculs/optimisation/c_modules
{
  date
  echo "===== make clean && make ====="
  make clean && make
  echo "===== test_illinois.py ====="
  python3 test_illinois.py
  echo "===== benchmark_illinois.py ====="
  python3 benchmark_illinois.py
} 2>&1 | tee ~/phase_c_test_log.txt
```

6. Suite après Phase C

Ordre	Action	Produit attendu
1	Sauvegarder Git avec git bundle --all.	Riemann_Lab_all_branches_2026-05-27.bundle
2	Nettoyer les fichiers d'audit hors repo.	git status sans archive 12 Go non suivie
3	Tester c_modules.	phase_c_test_log.txt
4	Tester compute_zeros_v4.py sur petit T.	run v4 T=80 ou T=300 propre
5	Comparer v3/v4.	tableau temps, zéros/s, précision, LMFDB
6	Générer pack scripts v2.	backup_all_branches.sh, restore_branch_C.sh, build_phase_c.sh, validate_run_v3.py, bootstrap_hprzeta_ubuntu24.sh

7. Scripts à générer dans la prochaine phase

- cleanup_audit_files_from_repo.sh : déplace automatiquement les gros fichiers d'audit hors du dépôt et ajoute les règles .gitignore.
- backup_all_branches.sh : crée bundle Git, logs de branches et archives légères.
- restore_branch_C.sh : restaure ou checkout Riemann_Lab_C après réinstallation.
- build_phase_c.sh : compile c_modules, exécute test_illinois.py et benchmark_illinois.py.
- validate_run_v3.py : vérifie nombre de lignes CSV, monotonie, doublons, LMFDB et cohérence log.
- bootstrap_hprzeta_ubuntu24.sh : prépare Ubuntu 24.04 avec paquets essentiels, Python, build-essential, libmpfr-dev, libgmp-dev, Git, Ollama léger et dossiers /mnt/data.

8. Rappel matériel pour guider les choix ce soir

- Machine : ASUS UX510UWK, Ubuntu 24.04.4 LTS, kernel 6.17.0-29-generic.
- CPU : Intel i7-7500U, 2 cœurs physiques / 4 threads, AVX2 disponible.
- RAM : environ 8 Go ; swap environ 15–16 Go.
- GPU : GTX 960M, 4 Go VRAM, driver NVIDIA 535.309.01, CUDA visible 12.2.
- Stockage : /mnt/data sur partition ext4 d'environ 662 Go, adaptée aux modèles, exports, audits et données validées.
- Conclusion opérationnelle : tests C/libmpfr et v4 possibles ; éviter gros LLM, embeddings massifs et batch GPU trop volumineux.

9. Check-list de reprise ce soir

- [] Vérifier que les PDF et fichiers texte sont bien dans C:\PerSoTestArmel\Zeta.
- [] Revenir sur le PC Linux.
- [] Déplacer l'archive audit 12 Go hors du repo.
- [] Ajouter les règles .gitignore.
- [] Créer git bundle --all.
- [] Checkout Riemann_Lab_C.
- [] Lancer make clean && make dans c_modules.
- [] Lancer test_illinois.py.
- [] Lancer benchmark_illinois.py.
- [] M'envoyer phase_c_test_log.txt.

10. Décision finale du complément

Le projet est prêt pour la reprise du soir. La priorité n'est plus la collecte d'audit système : cette partie est suffisamment documentée. La priorité devient la sécurisation Git, le nettoyage des fichiers non suivis, puis la validation reproductible de la Phase C.